Robotic Manipulation

Perception, Planning, and Control

Russ Tedrake

© Russ Tedrake, 2020-2026
Last modified 2026-9-13.
How to cite these notes, use annotations, and give feedback.

Note: These are working notes used for a course being taught at MIT. They will be updated throughout the Fall 2026 semester.

Search these notes

Search these notes...

PDF version of the notes

You can also download a PDF version of these notes (updated much less frequently) from here.

The PDF version of these notes are autogenerated from the HTML version. There are a few conversion/formatting artifacts that are easy to fix (please feel free to point them out). But there are also interactive elements in the HTML version are not easy to put into the PDF. When possible, I try to provide a link. But I consider the online HTML version to be the main version.

Table of Contents

- Preface
- Chapter 1: Introduction
  - Manipulation is more than pick-and-place
  - Open-world manipulation
  - Simulation
  - These notes are interactive
  - Model-based design and analysis
  - Organization of these notes
  - Exercises
- Chapter 2: Let's get you a robot
  - Robot description files
  - Arms
    - Position-controlled robots
      - Position Control.
      - An aside: link dynamics with a transmission.
    - Torque-controlled robots

        - A proliferation of hardware
        - Simulating the Kuka iiwa
    - Hands
        - Dexterous hands
        - Simple grippers
        - Soft/underactuated hands
        - Other end effectors
        - If you haven't seen it...
    - Sensors
    - Putting it all together
        - HardwareStation
        - HardwareStationInterface
        - HardwareStation stand-alone simulation
    - More HardwareStation examples
    - Exercises
- Chapter 3: Basic Pick and Place
    - Monogram Notation
    - Pick and place via spatial transforms
    - Spatial Algebra
        - Representations for 3D rotation
    - Forward kinematics
        - The kinematic tree
        - Forward kinematics for pick and place
    - Differential kinematics (Jacobians)
    - Differential inverse kinematics
        - The Jacobian pseudo-inverse
        - Invertibility of the Jacobian
    - Defining the grasp and pre-grasp poses
    - A pick and place trajectory
    - Putting it all together
    - Differential inverse kinematics with constraints
        - Pseudo-inverse as an optimization
        - Adding velocity constraints
        - Adding position and acceleration constraints
        - Joint centering
        - Tracking a desired pose
        - Alternative formulations
    - Exercises
- Chapter 4: Geometric Pose Estimation
    - Cameras and depth sensors
        - Depth sensors
        - Simulation
    - Representations for geometry

    - Point cloud registration with known correspondences
    - Iterative Closest Point (ICP)
    - Dealing with partial views and outliers
        - Detecting outliers
        - Point cloud segmentation
        - Generalizing correspondence
        - Soft correspondences
        - Nonlinear optimization
            - Precomputing distance functions
        - Global optimization
    - Non-penetration and "free-space" constraints
        - Free space constraints as non-penetration constraints
    - Tracking
    - Putting it all together
    - Looking ahead
    - Exercises
- Chapter 5: Bin Picking
    - Generating random cluttered scenes
        - Falling things
    - Static equilibrium with frictional contact
        - Spatial force
        - Collision geometry
        - Contact forces between bodies in collision
        - The Contact Frame
        - The (Coulomb) Friction Cone
        - Static equilibrium as an optimization problem
    - Contact simulation
    - Model-based grasp selection
        - The contact wrench cone
        - Colinear antipodal grasps
    - Grasp selection from point clouds
        - Point cloud pre-processing
        - Estimating normals and local curvature
        - Evaluating a candidate grasp
        - Generating grasp candidates
    - The corner cases
    - Programming the Task Level
        - State Machines and Behavior Trees
        - Task planning
        - Large Language Models
        - A simple state machine for "clutter clearing"
    - Putting it all together
    - Exercises

- Chapter 6: Motion Planning
  - Inverse Kinematics
    - From end-effector pose to joint angles
    - IK as constrained optimization
    - Global inverse kinematics
    - Inverse kinematics vs differential inverse kinematics
    - Grasp planning using inverse kinematics
  - Kinematic trajectory optimization
    - Trajectory parameterizations
    - Optimization algorithms
  - Sampling-based motion planning
    - Rapidly-exploring random trees (RRT)
    - The Probabilistic Roadmap (PRM)
    - Post-processing
    - Sampling-based planning in practice
  - Motion Planning w/ Graphs of Convex Sets (GCS)
    - Graphs of Convex Sets
    - GCS (Kinematic) Trajectory Optimization
    - Convex decomposition of (collision-free) configuration space
    - GcsTrajOpt Examples
    - Variations and Extensions
  - Time-optimal path parameterizations
  - Exercises
- Chapter 7: Mobile Manipulation
  - A New Cast of Characters
  - What's different about perception?
    - Partial views / active perception
    - Unknown (potentially dynamic) environments
    - Robot state estimation
  - What's different about motion planning?
    - Wheeled robots
      - Holonomic drives
      - Nonholonomic drives
    - Legged robots
  - What's different about simulation?
  - Navigation
    - Mapping (in addition to localization)
    - Identifying traversable terrain
  - Exercises
- Chapter 8: Manipulator Control
  - The Manipulator-Control Toolbox
  - Assume your robot is a point mass
    - Trajectory tracking

        - (Direct) force control
        - Indirect force control
        - Hybrid position/force control
    - The general case (using the manipulator equations)
        - Trajectory tracking
        - Joint stiffness control
        - Cartesian stiffness and operational space control
        - Some implementation details on the iiwa
    - Putting it all together
    - Peg in hole
    - Exercises
- Chapter 9: Object Detection and Segmentation
    - Getting to big data
        - Crowd-sourced annotation datasets
        - Segmenting new classes via fine tuning
        - Annotation tools for manipulation
        - Synthetic datasets
        - Self-supervised learning
        - Even bigger datasets
    - Object detection and segmentation
    - Putting it all together
    - Variations and Extensions
        - Pretraining wth self-supervised learning
        - Leveraging large-scale models
    - Exercises
- Chapter 10: Deep Perception for Manipulation
    - Pose estimation
        - Pose representation
        - Loss functions
        - Pose estimation benchmarks
        - Limitations
    - Grasp selection
    - (Semantic) Keypoints
    - Dense Correspondences
    - Scene Flow
    - Task-level state
    - Other perceptual tasks / representations
    - Exercises
- Chapter 11: Reinforcement Learning
    - RL Software
    - Policy-gradient methods
        - Black-box optimization
        - Stochastic optimal control

        - Using gradients of the policy, but not the environment
        - REINFORCE, PPO, TRPO
        - Control for manipulation should be easy
    - Value-based methods
    - Model-based RL
    - Exercises
- Chapter 12: Soft Robots and Tactile Sensing
    - Why soft?
    - Soft robot hardware
    - Soft-body simulation
    - Tactile sensing
        - What information do we want/need?
        - Visuotactile sensing
        - Whole-body sensing
        - Simulating tactile sensors
    - Perception with tactile sensors
    - Control with tactile sensors

Appendix

- Appendix A: Spatial Algebra
    - Position, Rotation, and Pose
    - Spatial velocity
    - Spatial force
- Appendix B: Drake
    - Pydrake
    - Online Jupyter Notebooks
        - Running on Google Colab
        - Enabling licensed solvers
    - Running on your own machine
    - Getting help
- Appendix C: DrakeGym Environments
    - Box Flipup
- Appendix D: Setting up your own "Manipulation Station"
    - Message Passing
    - Kuka LBR iiwa + Schunk WSG Gripper
    - Franka Panda
    - Intel Realsense D415 Depth Cameras
- Appendix E: Miscellaneous
    - How to cite these notes
    - Annotation tool etiquette
    - Some great final projects
    - Please give me feedback!

You can find documentation for the source code supporting these notes here.

Preface

I've always loved robots, but it's only relatively recently that I've turned my attention to robotic manipulation. I particularly like the challenge of building robots that can master physics to achieve human/animal-like dexterity and agility. It was passive dynamic walkers and the beautiful analysis that accompanies them that first helped cement this centrality of dynamics in my view of the world and my approach to robotics. From there I became fascinated with (experimental) fluid dynamics, and the idea that birds with articulated wings actually "manipulate" the air to achieve incredible efficiency and agility. Humanoid robots and fast-flying aerial vehicles in clutter forced me to start thinking more deeply about the role of perception in dynamics and control. Now I believe that this interplay between perception and dynamics is truly fundamental, and I am passionate about the observation that relatively "simple" problems in manipulation (how do I button up my dress shirt?) expose the problem beautifully.

My approach to programming robots has always been very computational/algorithmic. I started out using tools primarily from machine learning (especially reinforcement learning) to develop the control systems for simple walking machines; but as the robots and tasks got more complex I turned to more sophisticated tools from model-based planning and optimization-based control. In my view, no other discipline has thought so deeply about dynamics as has control theory, and the algorithmic efficiency and guaranteed performance/robustness that can be obtained by the best model-based control algorithms far surpasses what we can do today with learning control. Unfortunately, the mathematical maturity of controls-related research has also led the field to be relatively conservative in their assumptions and problem formulations; the requirements for robotic manipulation break these assumptions. For example, robust control typically assumes dynamics that are (nearly) smooth and uncertainty that can be represented by simple distributions or simple sets; but in robotic manipulation, we must deal with the non-smooth mechanics of contact and uncertainty that comes from varied lighting conditions, and different numbers of objects with unknown geometry and dynamics. In practice, no state-of-the-art robotic manipulation system to date (that I know of) uses rigorous control theory to design even the low-level feedback that determines when a robot makes and breaks contact with the objects it is manipulating. An explicit goal of these notes is to try to change that.

In the past few years, deep learning has had an unquestionable impact on robotic perception, unblocking some of the most daunting challenges in performing manipulation outside of a laboratory or factory environment. We will discuss relevant tools from deep learning for object recognition, segmentation, pose/keypoint estimation, shape completion, etc. Now relatively old approaches to learning control are also enjoying an incredible surge in popularity, fueled in part by massive computing power and increasingly available robot hardware and simulators. Unlike

learning for perception, learning control algorithms are still far from a technology, with some of the most impressive looking results still being hard to understand and to reproduce. But the recent work in this area has unquestionably highlighted the pitfalls of the conservatism taken by the controls community. Learning researchers are boldly formulating much more aggressive and exciting problems for robotic manipulation than we have seen before -- in many cases we are realizing that some manipulation tasks are actually quite easy, but in other cases we are finding problems that are still fundamentally hard.

Finally, it feels that the time is ripe for robotic manipulation to have a real and dramatic impact in the world, in fields from logistics to home robots. Over the last few years, we've seen UAVs/drones transition from academic curiosities into consumer products. Even more recently, autonomous driving has transitioned from academic research to industry, at least in terms of dollars invested. Manipulation feels like the next big thing that will make the leap from robotic research to practice. It's still a bit risky for a venture capitalist to invest in, but nobody doubts the size of the market once we have the technology. How lucky are we to potentially be able to play a role in that transition?

So this is where the notes begin... we are at an incredible crossroads between learning and control and robotics with an opportunity to have immediate impact in industrial and consumer applications and potentially even to forge entirely new eras for systems theory and controls. I'm just trying to hold on and to enjoy the ride.

A MANIPULATION TOOLBOX

Another explicit goal of these lecture notes is to provide high-quality implementations of the most useful tools in a manipulation scientist's toolbox. When I am forced to choose between mathematical clarity and runtime performance, the clear formulation is always my first priority; I will try to include a performant formulation, too, if possible or try to give pointers to alternatives. Manipulation research is moving quickly, and I aim to evolve these notes to keep pace. I hope that the software components provided in DRAKE and in these notes can be directly useful to you in your own work.

If you would like to replicate any or all of the hardware that we use for these notes, you can find information and instructions in the appendix.

As you use the code, please consider contributing back (especially to the mature code in DRAKE). Even questions/bug reports can be important contributions. If you have questions/find issues with these notes, please submit them here.

First chapter

Accessibility © Russ Tedrake, 2020-2026